# RAHEE

## Test Phase 2 Report

| | |
|---|---|
| **Project** | RAHEE, an online digital version of a travel board game |
| **Team** | Team 31 (5 members) |
| **Report period** | Test Phase 2 (30 October - 5 November 2026) |
| **Build tested** | main branch, 3 November 2026 (Test Phase 1 build + the Phase 2 tests and fixes) |

### Summary

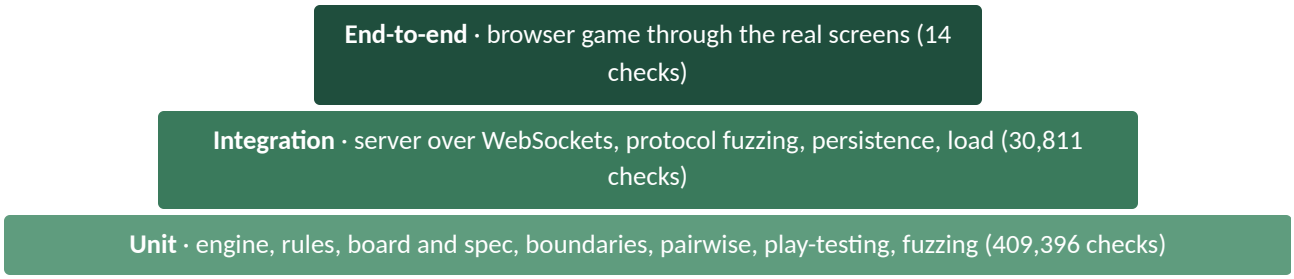
In Test Phase 2 we added seven new test scripts and three new rule scenarios, built around the techniques from the CS 202 testing lecture: specification testing, equivalence partitioning, boundary-value analysis, pairwise testing, fuzz testing (including mutation-based fuzzing of the network messages), load testing, end-to-end testing in a real browser, and code coverage. The full suite now runs **440,221 checks, and all of them pass** on the final build. Line coverage rose from 93.8% to 96.4% and branch coverage from 82.4% to 93.1%.

The new tests found **four real defects**, all now fixed. The most serious one let a player collect extra memories (and so extra points) by sending "finish move" again after a move had ended, or end a turn without moving.

| Required item | Answer (details in the section shown) |
|---|---|
| 1. Test categories included | 12 categories in a testing pyramid: unit and data, regression, rule scenarios, specification conformance, boundary values and equivalence classes, pairwise, simulated play-testing, fuzzing and robustness, persistence, server end-to-end with protocol fuzzing, load, and browser end-to-end (Section 1) |
| 2. Number of tests added | 380,877 new checks in 7 new test scripts and 3 new rule scenarios. With the Phase 1 tests, 440,221 checks in total (Section 2) |
| 3. Number of tests passed | 440,221 of 440,221 (Section 3) |
| 4. Number of tests failed | 0 on the final build. During the phase the new tests failed on the old code and found 4 defects (D1 to D4), which were fixed (Section 4) |
| 5. Tools used and their outputs | Node.js 20 test scripts, the computer players, ws WebSocket client, in-memory database, Playwright 1.64 with Chromium, c8 code coverage, GitHub Actions (Section 5) |

## 1. Test Categories Included

The tests follow the testing pyramid from the lecture: many fast unit tests at the bottom, fewer integration tests, and a small number of slow end-to-end tests at the top.




| # | Category | Lecture technique | What it checks | File |
|---|---|---|---|---|
| 1 | Unit and data | Unit testing | Board data, deals for 2 to 6 players, scoring examples from the manual | test-engine.mjs |
| 2 | Regression | Regression testing | Week 4 bugs B1 to B4 stay fixed | test-engine.mjs |
| 3 | Rule scenarios | Specification testing | One hand-built situation per rule in RULES.md. **3 new** in Phase 2 for the order of actions in a turn | test-rules.mjs |
| 4 | Specification conformance **(new)** | Specification testing; verification | Reads the money, scoring, deck and character tables from RULES.md itself and compares them with board.json. Checks the board structure. Compares the engine's route search with an independent breadth-first search on every pair of cities and every card value 1 to 10 | test-board.mjs |
| 5 | Boundary values and equivalence classes **(new)** | Boundary-value analysis; equivalence partitioning | Each card value at k-1, k and k+1 steps; over-budget penalty around ₹20,000 and each ₹500; memory sets; the 5-destination bonus; 1 to 7 players. One case from each class of route (road, flight, each ship) and of Ace use | test-boundaries.mjs |
| 6 | Pairwise **(new)** | Pairwise testing | Number of players × bot level × first player × characters: 21 games cover all 83 pairs of values (180 games to try every combination). Each is played to the end with the rules checked after every move | test-boundaries.mjs |
| 7 | Simulated play-testing | Validation | 300 complete games between computer players, every move checked | playtest.mjs |

| | | | | |
|---|---|---|---|---|
| 8 | Fuzzing and robustness **(new)** | Fuzz testing | During 60 real games, before every move: out-of-turn moves, random broken or hostile moves, unknown action names, "end" actions at every stage of the turn, and a replay of the move just played. Each must be refused with a clear message and leave the game exactly as it was. Also checks that the same seed gives the same game | test-robustness.mjs |
| 9 | Persistence **(new)** | Integration testing | A server restart is simulated after every move of 40 games (8,029 restarts). The game, seats and reconnection tokens must come back identical, and the game must play on to the end | test-persistence.mjs |
| 10 | Server end-to-end and protocol fuzzing **(new)** | Integration testing; mutation-based fuzzing | Lobby limits (name length 17/18/19, 6 seats then "room is full"), reconnection edge cases, 4,000 mutated messages plus raw binary and a 2 MB message (the server must not crash), and "../" path tricks on the web server. Also the existing full 3- and 6-player server games | test-protocol.mjs, test-server.mjs |
| 11 | Load and performance **(new)** | Non-functional testing | 20 games at the same time on one server: all must finish, hands stay private, and 95% of moves must be answered within 500 ms | test-load.mjs |
| 12 | Browser end-to-end **(new)** | System / E2E testing | A full 3-player game through the real screens in Chromium: rules sheet, lobby, character, deal, map panel, Auto play, a page reload mid-game, Journey's End, and no browser errors | test-browser.cjs |

Following the lecture's "pesticide paradox", the random tests do not only re-run old cases: the fuzzers produce new inputs every run, and the play-tests can be re-run with a different seed.

## 2. Number of Tests Added

A "check" is one test condition that can pass or fail. The counts are from the run on our machine. The counts for the server, persistence and load tests change slightly from run to run because their games are random.

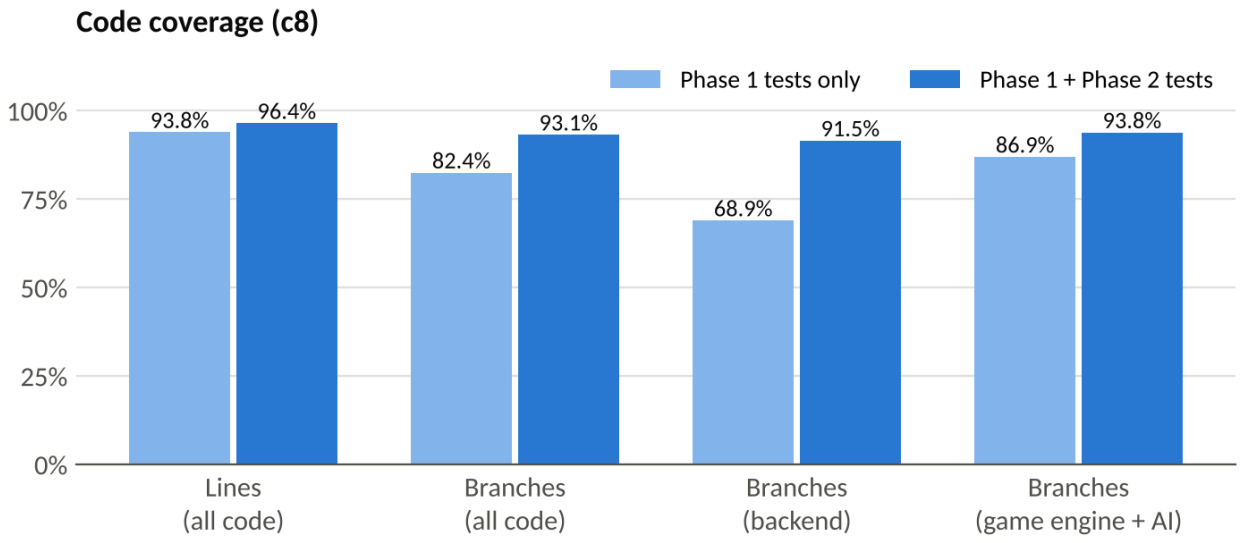
| Suite | What was added | Checks |
|---|---|---|
| Specification conformance (test-board.mjs) | RULES.md tables 246, board structure 1,493, route search on every pair 101,000 | 102,739 |
| Boundaries and pairwise (test-boundaries.mjs) | Boundary and class cases 106, 21 pairwise games 18,153 | 18,259 |
| Fuzzing and robustness (test-robustness.mjs) | Bad actions during 60 games 232,746, determinism 231 | 232,977 |
| Persistence (test-persistence.mjs) | 8,029 simulated restarts, seat reclaiming | 16,393 |
| Protocol (test-protocol.mjs) | Lobby 16, reconnection 17, fuzzing 5, static files 11 | 49 |
| Load (test-load.mjs) | 20 games at once | 10,443 |
| Browser (test-browser.cjs) | One full game through the screens | 14 |
| Rule scenarios (test-rules.mjs) | 3 new scenarios for the order of actions in a turn | 3 |
| **Added in Phase 2** | **7 new scripts + 3 rule scenarios** | **380,877** |
| Phase 1 tests (still run) | Engine 5,499, rules 31, play-test 49,888, server 781 + 3,145 | 59,344 |
| **Total** | | **440,221** |

## 3. Number of Tests Passed

| Suite (command) | Passed | Failed | Result |
|---|---|---|---|
| Engine: unit, regression, 80 random games | 5,499 | 0 | Pass |
| Rule scenarios | 34 | 0 | Pass |
| AI play-testing, 300 games | 49,888 | 0 | Pass |
| Server end-to-end, 3 players | 781 | 0 | Pass |
| Server end-to-end, 6 players | 3,145 | 0 | Pass |
| Specification conformance (`npm run test:board`) | 102,739 | 0 | Pass |
| Boundaries and pairwise (`npm run test:boundaries`) | 18,259 | 0 | Pass |
| Fuzzing and robustness (`npm run test:robustness`) | 232,977 | 0 | Pass |
| Persistence (`npm run test:persistence`) | 16,393 | 0 | Pass |
| Protocol (`npm run test:protocol`) | 49 | 0 | Pass |
| Load, 20 games at once (`npm run test:load`) | 10,443 | 0 | Pass |
| Browser end-to-end (`npm run test:browser`) | 14 | 0 | Pass |
| **Total** | **440,221** | **0** | **All pass** |

### Code coverage

Coverage shows how much of the code the tests actually run. We measured it with c8 over the whole `npm test` run, including the server processes it starts.



| File | Lines | Branches | Not covered (why) |
|---|---|---|---|
| backend/actions.js, protocol.js, handlers.js, rooms.js, redact.js | 100% | 87.5-100% | - |
| backend/app.js | 91.7% | 73.9% | Start-up messages for the local network, and the error path when the database cannot be reached |
| backend/db.js | 87.9% | 90.9% | The real MySQL/TiDB connection, which the tests replace with the in-memory database |
| backend/bots.js | 78.4% | 85.7% | Rare timing paths when a computer player waits for a rider |
| frontend/src/engine.js | 97.2% | 94.2% | Save and load in the browser's local storage, which only runs inside a browser |
| frontend/src/ai.js | 98.2% | 93.0% | A helper used only by the play-test report |
| **All files** | **96.4%** | **93.1%** | **Functions 94.8%** |

The "Phase 1 tests only" figures were measured the same way, by running only the five Phase 1 suites on the Phase 2 build.

## 4. Number of Tests Failed

**0 tests fail on the final build.** While Phase 2 was being written, the new tests failed on the old code and found four real defects. Each was fixed, and the test that found it passes now and stays in the suite so the defect cannot come back.

| ID | Defect | Found by | Fix | Severity |
|---|---|---|---|---|
| D3 | The "end" actions did not check when they were sent. Sending "finish move" again after a move had ended gave the player another memory each time; "finish road trip" again gave every traveller another road-trip memory; and "end turn" was accepted after moving 0 steps, which breaks the rule "You cannot pass" | Robustness test ("end" actions at every stage, and replaying each move) | Each of the three actions now checks the stage of the turn first and refuses with a clear message, e.g. "finish your move before ending your turn" (frontend/src/engine.js) | High |

| | | | | |
|---|---|---|---|---|
| D1 | The server accepted "constructor", "toString" and "hasOwnProperty" as game moves, because the action names were looked up on a plain JavaScript object. The game state was not changed, but the moves were treated as valid | Robustness test; confirmed over the network by the protocol test | Only the game's own action names are accepted (backend/actions.js, one line) | Medium |
| D2 | Some moves sent at the wrong moment were refused with an internal error such as "Cannot read properties of null" instead of a clear message (901 times in one run). The game was not changed | Robustness test | Clear messages such as "play a card before finishing a move" and "there is no road trip under way to finish" (frontend/src/engine.js) | Low |
| D4 | The top bar said "Their budget left" on the player's own turn | Browser test | Says "Your budget left" on your own turn (frontend/src/main.js); the browser test checks it | Low |

### The same tests on the old code

| Test | On the old code | On the final build |
|---|---|---|
| Robustness (test-robustness.mjs) | 21,839 of 223,510 checks failed: "end" actions accepted at the wrong stage, replayed moves handing out extra memories, internal error messages | **232,977 of 232,977 pass** |
| Rule scenarios (test-rules.mjs) | 3 of 34 failed: the three new "order of actions" scenarios | **34 of 34 pass** |
| Protocol (test-protocol.mjs) | 4 of 49 failed: "constructor", "toString", "__proto__" and "hasOwnProperty" not refused as unknown actions | **49 of 49 pass** |

### Observation to confirm with the POC

The board has a third ship route, Agatti-Kavaratti inside Lakshadweep, with no surcharge. RULES.md lists only the Kochi-Kavaratti (₹1,000) and Sri Vijayapuram-Great Nicobar (₹200) ships. The specification test accepts this one hop as it is in the board data, and we will confirm it with the POC.

## 5. Tools Used and Their Outputs

| Tool | Used for |
|---|---|
| Node.js 20 test scripts (no test framework) | All suites. Each script counts its checks and ends with an error code if any fails. `npm test` (tools/run-tests.mjs) runs all eleven |
| Computer players (frontend/src/ai.js) | Play the games in the play-test, pairwise, robustness, persistence, server, load and browser tests, from the same view a player's browser gets |
| ws (WebSocket client) and the in-memory database (DB_DRIVER=memory) | Server, protocol and load tests talk to the real backend exactly as a browser does, without needing MySQL |
| Our own fuzzers (in test-robustness.mjs and test-protocol.mjs) | Random and hostile moves; radamsa-style mutation of network messages (bit flips, deleted or repeated chunks, JSON symbols, protocol keywords, interesting values) |
| Playwright 1.64 with Chromium 156 | Browser end-to-end test: real clicks, a page reload, browser-error capture, screenshots |
| c8 10.1 (V8 code coverage) | `npm run coverage`: line, branch and function coverage of frontend/src and backend |
| GitHub Actions | Runs `npm test` on every push |

### Output: npm test (Phase 2 suites; the Phase 1 suites print as in the Phase 1 report)

```
=== rules ===
  ...
Order of actions in a turn
  ok   finishing a move twice does not give a second memory
  ok   a turn cannot end without moving at least one step
  ok   finishing a road trip twice does not give second road-trip memories
Budget and scoring
  ok   starting budget is ₹20,000 (RULES.md)
  ok   manual turn example: Jaipur, 5♣ to Gwalior costs ₹1,000 + ₹2,000 stay
34 passed, 0 failed

=== board ===
Checks by section:
  RULES.md money table vs board.json.............................      10
  RULES.md scoring table vs board.json...........................       6
  RULES.md deck table vs the deal................................     211
  RULES.md character cards vs board.json.........................      19
  Board structure................................................   1,490
  Every city reachable; only the islands need a flight or ship....       3
  Route search vs independent BFS (all pairs, card values 1-10)... 101,000
  Total.......................................................... 102,739
all 102,739 checks passed

=== boundaries ===
pairwise: 21 games cover all 83 pairs of setting values (exhaustive testing would need 180 games)
Checks by section:
  Boundary: movement for every card value (k-1, k, k+1 steps)...      53
  Boundary: over-budget penalty around ₹20,000 and each ₹500....      10
  Boundary: memory sets around 3 of a suit and 1 of each suit...      12
  Boundary: destination bonus around 5 completed.................      10
  Boundary: player counts 1 to 7.................................       7
  Equivalence classes: route kinds and Ace uses..................      14
  Pairwise: all-pairs game settings, each played to the end.....  18,153
  Total..........................................................  18,259
all 18,259 checks passed

=== robustness ===
60 games played with bad input before every move
  29,715 "end" actions at every stage, 10,127 replayed moves
  29,149 out-of-turn actions, 59,384 fuzzed actions refused, 1,738 fuzzed actions that happened to be
legal
Checks by section:
  Bad actions during real games (refused, game unchanged)....... 232,746
  Determinism (same seed, same deal; same moves, same end)......     231
  Total.......................................................... 232,977
all 232,977 checks passed

=== persistence ===
40 games, 7,989 moves, 8,029 simulated server restarts
Checks by section:
  Restart after every move: game identical, seats and tokens kept  16,386
  Reclaiming a seat after a restart..............................       7
  Total..........................................................  16,393
all 16,393 checks passed

=== protocol ===
fuzzing: 4,000 mutated messages (80 raw binary frames) + one 2 MB message, 4,157 replies; server still
running: yes
Checks by section:
  Lobby rules and limits.........................................      16
  Reconnection and seat takeover.................................      17
  Mutation-based fuzzing of the message protocol.................       5
  Static file server.............................................      11
  Total..........................................................      49
```

```
all 49 checks passed

=== load (20 games at once) ===
20 games at once (4 players each: 1 WebSocket client + 3 computer players)
  finished: 20/20 in 1.9 s
  client moves: 1,015, state messages received: 3,453 (1,793 per second)
  response time per move: median 3.8 ms, 95th percentile 27.4 ms, worst 82.9 ms
  server memory: 65 MB before, 143 MB after
all 10,443 checks passed

=== summary ===
  PASS  engine  (0.1s)
  PASS  rules  (0.1s)
  PASS  playtest  (7.1s)
  PASS  server (3 players)  (1.9s)
  PASS  server (6 players)  (2.6s)
  PASS  board  (3.8s)
  PASS  boundaries  (0.7s)
  PASS  robustness  (16.4s)
  PASS  persistence  (3.4s)
  PASS  protocol  (7.1s)
  PASS  load (20 games at once)  (4.2s)
```

### Output: browser end-to-end test (npm run test:browser)

```
  ok   landing page loads with name field and Create button
  ok   "How to play" opens the one-page rules sheet
  ok   the rules sheet closes with Escape
  ok   room created and a 4-letter room code shown — Q2SL
  ok   host can add two computer players
  ok   character card can be picked; host can deal
  ok   cards dealt: setup screen asks to lay a road-trip card
       [7s] round 1
  ok   top bar says "Your budget left" on your own turn
       [15s] round 2
  ok   "Map & destinations" panel opens and lists the other players
       [46s] round 3
  ok   page reload mid-game returns to the same seat
       [49s] round 4
  ok   "Auto" let the computer play our seat
  ok   game played to the Journey's End scoreboard — 61s
  ok   scoreboard lists all three travellers
  ok   no browser errors during the whole game
14 passed, 0 failed
```

### Output: code coverage (npm run coverage)

```
--------------|---------|----------|---------|---------|-------------------------------------
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------------|---------|----------|---------|---------|-------------------------------------
All files     |    96.4 |    93.13 |   94.81 |    96.4 |
 backend      |   94.55 |    91.48 |   96.55 |   94.55 |
  actions.js  |     100 |      100 |     100 |     100 |
  app.js      |   91.66 |    73.91 |     100 |   91.66 | 62-66,93-95
  bots.js     |    78.4 |    85.71 |   83.33 |    78.4 | 69-75,88-90,109-125
  db.js       |   87.87 |     90.9 |   66.66 |   87.87 | 41-48
  handlers.js |     100 |    98.64 |     100 |     100 | 63
  protocol.js |     100 |      100 |     100 |     100 |
  redact.js   |     100 |     87.5 |     100 |     100 | 20
  rooms.js    |     100 |    89.47 |     100 |     100 | 58,70,112,115,120,146,156
 frontend/src |   97.48 |    93.77 |    93.5 |   97.48 |
  ai.js       |   98.15 |    92.98 |      95 |   98.15 | 345,425-431
  engine.js   |   97.16 |    94.24 |   92.98 |   97.16 | 641-643,792-795,896-898,900-913,915-916
--------------|---------|----------|---------|---------|-------------------------------------
Statements   : 96.4% ( 2062/2139 )
Branches     : 93.13% ( 936/1005 )
Functions    : 94.81% ( 128/135 )
Lines        : 96.4% ( 2062/2139 )
```

## Screenshots from the browser test

1. "Map & destinations" panel open during the game, with "Your budget left" in the top bar



2. Journey's End scoreboard reached in the browser




## 6. Examples from the Test Code and the Fixes

**Specification test: route search compared with an independent breadth-first search** (tools/test-board.mjs)

```
for (const a of ids) for (const b of ids) {
  const dAll = distAll.get(a).get(b), dRoad = distNoFlight.get(a).get(b) ?? Infinity;
  for (let k = 1; k <= 10; k++) {
    const p = E.shortestPath(graph, a, b, k);
    if (dAll > k) { check(p === null, `${label}: found a route although the shortest is ${dAll}
steps`); continue; }
    ...
    if (dRoad <= k) ok = ok && flights === 0 && steps === dRoad; // B2: roads first, and the shortest
road route
    else ok = ok && steps === dAll;
```

**Boundary values: every card value moves exactly k steps, not k+1** (tools/test-boundaries.mjs)

```
const cases = [["A", 1], ["2", 2], ["3", 3], ["4", 4], ["5", 5], ["6", 6], ["10", 10], ["J", 10],
["Q", 5], ["K", 5]];
for (const [rank, k] of cases) {
  ...
  check(throws(() => E.endMovement(s)), `${rank}: ending a move after 0 steps was allowed`);
  for (let i = 0; i < k; i++) ok = ok && !throws(() => E.stepTo(s, graph, longRoad.edges[i].to));
  check(throws(() => E.stepTo(s, graph, longRoad.edges[k].to)), `${rank}: step ${k + 1} was allowed`);
```

**Fuzzing: a refused move must leave the game exactly as it was** (tools/test-robustness.mjs)

```
const copy = copyOf(s);
try { applyAction(copy, graph, at.seat, at.name, at.args); } catch (e) { err = e; }
...
check(!(err instanceof TypeError), `"${at.name}" was refused with an internal error instead of a clear
message`);
check(snapshot(copy) === before, `refused "${at.name}" (${err.message}) but changed the game anyway`);
```

**Fix for D3** (frontend/src/engine.js)

```
 export function endMovement(s) {
   const t = s.turn;
+  if (!t) throw new Error("play a card before finishing a move");
+  if (t.stage !== "moving") throw new Error("you are not moving right now — this move has already
been finished");
 ...
 export function endTurn(s) {
   const t = s.turn;
+  if (!t) throw new Error("play a card and move before ending your turn");
+  if (t.stage !== "resolve") throw new Error("finish your move before ending your turn");
```

**Fix for D1** (backend/actions.js)

```
-  const fn = ACTIONS[name];
+  const fn = Object.hasOwn(ACTIONS, name) ? ACTIONS[name] : null; // not "constructor", "toString",
...
   if (!fn) throw new Error(`unknown action "${name}"`);
```

## 7. How to Re-run the Tests

```
cd backend && npm install && cd ..     # once
npm test                               # all 11 suites (about 1 minute)
npm run coverage                       # the same, with code coverage
npm install --no-save playwright && npx playwright install chromium    # once, for the browser test
npm run test:browser                   # full game through the screens (about 1 minute)
```

## 8. Date of the Current Build

| | |
|---|---|
| **Build tested** | 3 November 2026, main branch |
| **Changed in this phase** | New: tools/test-board.mjs, test-boundaries.mjs, test-robustness.mjs, test-persistence.mjs, test-protocol.mjs, test-load.mjs, test-browser.cjs. Updated: tools/test-rules.mjs (3 scenarios), tools/run-tests.mjs, package.json (test commands), .gitignore. Fixes: frontend/src/engine.js (D2, D3), backend/actions.js (D1), frontend/src/main.js (D4), backend/app.js (clean shutdown, so coverage includes the server) |